WoSM Academy

CS102-F26 · DIGITAL DESIGN AND COMPUTER ARCHITECTURE I

# The Clock and Latches

Lecture 11: the clock, memory, the SR latch and the D latch

Dr. Ali Khudiyev

Fall 2026



CC BY-SA 4.0

# Today


01 The clock

02 Memory

03 The SR latch

04 The D latch

05 Summary

01

# The clock

# A clock for a computer

A clock orchestrates all of us at once. Inside a computer, a clock orchestrates all the parts. Its face has only two numbers: 0 and 1.



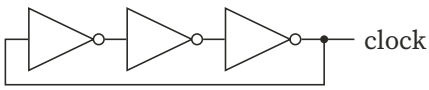

- The period $T$ is the time for one full cycle; the frequency is $f = 1/T$, in hertz (cycles per second).
- A *rising* edge goes from 0 to 1, a *falling* edge from 1 to 0. The *duty cycle* is the share of the period spent at 1, usually 50%.

# Periods and frequencies

| Period $T$ | Frequency $f = 1/T$ | |
|---|---|---|
| 10 µs | 100 kHz | the clock in the book's Figure 5.7 |
| 5 ns | 200 MHz | the speed limit of Lecture 09's example circuit |
| 0.33 ns | 3 GHz | a typical processor today |

> **Remark**
>
> Every part of a synchronous circuit does its work between two ticks. So the clock can tick only as fast as the slowest part allows: a period shorter than its propagation delay gives wrong results (Unit 14).

# Making a clock




a ring of three NOT gates

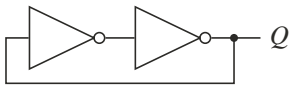
- The book's clock is one NOT gate whose output feeds its own input: the output can never settle, so it keeps flipping.
- In a real circuit we use a ring of an odd number $n \geq 3$ of NOT gates. Each half period takes $n$ gate delays, so $T = 2\,n\,t_{pd}$.
- Computers take their clock from a quartz crystal, and multiply its frequency on the chip.

With an even number of NOT gates the ring settles and never flips: remember this for the next slide.

02

# Memory

# What is memory?




two NOT gates in a ring

- Memory is something that persists while everything around it changes.
- Two NOT gates in a ring keep their value forever. But they keep whatever they started with: there is no way to put a value in.
- Better: memory is something that is updated in certain situations and kept in all others.

# A memory with two inputs

Give the circuit two inputs, (memorize-1, memorize-0):

- (1, 0): remember 1; (0, 1): remember 0; (0, 0): keep what is stored;
- (1, 1) makes no sense: one bit cannot remember 1 and 0 at once.

| Time step | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| Input | (1, 0) | (0, 0) | (0, 0) | (0, 1) | (0, 0) |
| Output | 1 | 1 | 1 | 0 | 0 |

**It is the mystic box**

Lecture 10's box with *On* as memorize-1, *Off* as memorize-0, and green as 1. We found $Q^+ = On + Off' \, Q$.

# Set, reset, hold

| $S$ | $R$ | $Q_{t+1}$ | $Q'_{t+1}$ |
|---|---|---|---|
| 0 | 0 | $Q_t$ | $Q'_t$ |
| 0 | 1 | 0 | 1 |
| 1 | 0 | 1 | 0 |
| 1 | 1 | not allowed | |

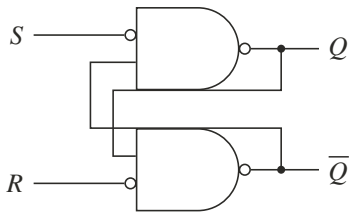
`S: set (memorize-1); R: reset (memorize-0)`

- The output is also given as a complement pair, $Q$ and $Q'$.
- Input (0, 0) gives 0 at some times and 1 at others, depending on the past. So no combinational circuit can do this: the circuit must be sequential.

03

# The SR latch

# The SR latch




- Two NAND gates, each fed by the other's output. The bubbles on $S$ and $R$ are NOT gates.
- The top gate outputs what is stored, $Q$; the bottom one its opposite, $\overline{Q}$:

$$Q = \text{NAND}(S', \overline{Q})$$
$$\overline{Q} = \text{NAND}(R', Q)$$

A NAND outputs 0 only when both inputs are 1. So a 0 on either input forces its output to 1.

# Setting and holding

**Set:** $S = 1$, $R = 0$, stored $Q = 0$.

1. $S' = 0$, so the top NAND outputs $Q = 1$.
2. Now the bottom NAND sees $R' = 1$ and $Q = 1$: $\overline{Q} = 0$.
3. The top NAND sees $S' = 0$ and $\overline{Q} = 0$: $Q$ stays 1. Stable.

**Hold:** then $S = R = 0$, stored $Q = 1$.

1. Both NANDs now see a 1 from their inverted input.
2. Top: $\text{NAND}(1, \overline{Q} = 0) = 1$. Bottom: $\text{NAND}(1, Q = 1) = 0$.
3. Nothing changes: the latch keeps $Q = 1$.

> **Remark**
>
> With $S = R = 0$ each gate passes on the complement of the other's output: the latch is the two-NOT ring of the last section. $S$ and $R$ are the way to write it.

# The full truth table

| $S$ | $R$ | $Q_t$ | $Q_{t+1}$ | $\overline{Q}_{t+1}$ | |
|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 1 | hold |
| 0 | 0 | 1 | 1 | 0 | hold |
| 0 | 1 | 0 | 0 | 1 | reset |
| 0 | 1 | 1 | 0 | 1 | reset |
| 1 | 0 | 0 | 1 | 0 | set |
| 1 | 0 | 1 | 1 | 0 | set |
| 1 | 1 | 0 | 1 | 1 | not valid: $Q = \overline{Q}$ |
| 1 | 1 | 1 | 1 | 1 | not valid: $Q = \overline{Q}$ |

Check each row yourself by pushing the inputs through the circuit, as in the book.

# The forbidden input

With $S = R = 1$ both outputs are 1. That alone is wrong, but worse comes when both inputs go back to 0:

| Released first | Result |
|---|---|
| $S$ | $R$ is still 1 for a moment: the latch resets, $Q = 0$ |
| $R$ | $S$ is still 1 for a moment: the latch sets, $Q = 1$ |
| both at exactly the same time | no input wins: $Q$ may flip back and forth |

- The input released last wins, and which one that is depends on tiny differences in timing.
- Stuck between 0 and 1 for a while is called metastability.
- The rule: never apply $S = R = 1$.

Built from two NOR gates instead, the latch works the same, except that $S = R = 1$ makes both outputs 0.

04

# The D latch

# The D latch

| $D$ | $E$ | $Q_{t+1}$ |
|---|---|---|
| 0 | 0 | $Q_t$ |
| 1 | 0 | $Q_t$ |
| 0 | 1 | 0 |
| 1 | 1 | 1 |

**Definition (D latch)**

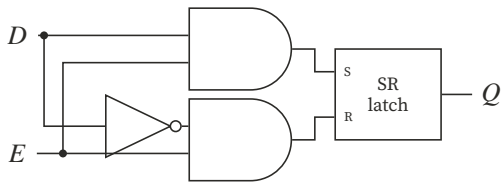
A *D latch* has a data input $D$ and an enable input $E$. While $E = 1$ it stores $D$; while $E = 0$ it keeps what it has.

- While $E = 1$, the output follows $D$: the latch is transparent.
- While $E = 0$ it is opaque: $D$ can change freely, $Q$ does not.

# From an SR latch to a D latch

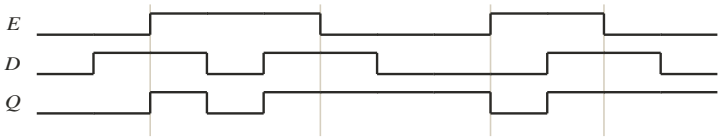
| $D$ | $E$ | $S$ | $R$ | $Q_{t+1}$ |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | $Q_t$ |
| 1 | 0 | 0 | 0 | $Q_t$ |
| 0 | 1 | 0 | 1 | 0 |
| 1 | 1 | 1 | 0 | 1 |

$S = DE, \qquad R = D'E$



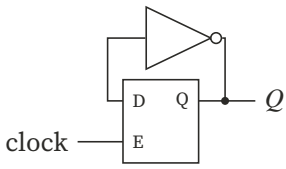

- The book's view: put a small 'interface' in front of the SR latch that turns $(D, E)$ into $(S, R)$.
- $S$ and $R$ can never both be 1, since $D$ and $D'$ never are: the D latch has no invalid inputs.

# The D latch in time




- While $E = 1$, $Q$ copies every change of $D$, even several in a row.
- When $E$ falls, $Q$ keeps the last value of $D$: 1 after the first window, 1 after the second.
- While $E = 0$, $D$ changes without effect.

# The trouble with transparency




- We want a bit that flips once per clock tick: $D = Q'$, enabled by the clock.
- But while the clock is 1 the latch is transparent, and the loop is a ring oscillator: $Q$ flips again and again.
- We need a memory that changes only at the instant of a clock edge.

**Next lecture**

The flip-flop: built from two D latches, it reacts to the clock edge, not to the clock level.

# Try it

1. What is the frequency of a clock with a period of 2 ns?
   $f = 1/2\,\text{ns} = 500\,\text{MHz}$.
2. A ring of 5 NOT gates, each with a delay of 20 ps. What period, and what frequency?
   $T = 2 \times 5 \times 20\,\text{ps} = 200\,\text{ps}$, so $f = 5\,\text{GHz}$.
3. An SR latch holds $Q = 1$. Apply $S = 0$, $R = 1$, then $S = R = 0$. What is $Q$?
   0: the reset stores 0, and the hold keeps it.
4. A D latch has $E = 1$ while $D$ goes 0, 1, 0; then $E = 0$ and $D = 1$. What is $Q$ at the end?
   0, the value of $D$ when $E$ fell.

05

# Summary

# Summary

- A clock is a signal that flips between 0 and 1 with period $T$ and frequency $f = 1/T$. A ring of an odd number of NOT gates makes one.
- Memory must be writable: update in some situations, hold in all others.
- The SR latch: set, reset, hold. $S = R = 1$ is not allowed, and releasing it can leave the latch metastable.
- The D latch: $Q$ follows $D$ while $E = 1$, and holds while $E = 0$. It has no invalid inputs.
- Transparency is a problem when a latch's output feeds back to its input: we need edge-triggered memory.

# Before the next lecture

- Read Sections 5.2, 5.3, 5.3.1 and 5.3.2 of the book.
- In Logisim-evolution, build the SR latch from NAND and NOT gates, and check every row of its truth table.
- Build a D latch on top of it, and drive $E$ from a clock. Watch $Q$ follow $D$ while the clock is 1.

> **Next lecture** (Unit 12)
>
> Flip-flops: the master–slave D flip-flop, asynchronous reset, and T and JK flip-flops.